

Dota Companion App

Product requirements and implementation brief for Claude Code | Version 1.0 | 29 September 2026

Working title: Ancient Atlas (placeholder). A unified Dota 2 companion for personal and group progression, patch intelligence, and Captain’s Mode-style draft practice. This document is a build specification, not a claim that all data is available from public APIs.

Recommended stack: Next.js App Router + TypeScript + MongoDB Atlas + shadcn/ui + Tailwind CSS + React Hook Form + Zod. One deployable application, with server-only integrations and clear domain boundaries.

1. Product goals and boundaries

- Primary audience: ranked players who play solo and with friends, review patches, and want structured draft practice.
- Distinctive loop: see patch changes relevant to your hero pool → practice a draft with friends → review shared matches → set the next experiment.
- Success measure: users return after a new patch and after a group session; draft rooms finish; players correctly understand solo/party results and data confidence.
- V1 exclusions: live in-match advantage overlay, automatic exact MMR reconstruction, unsupported predictive win percentages, or promises that all matches are public.

2. Feature inventory and release plan

Phase	Deliverables
Foundation (M0)	Repo, CI, design system, Steam sign-in, account linking, Mongo models and indexes, external data adapters, observability.
MVP (M1)	Player dashboard; recent matches; solo/party/unknown split; hero and role filters; manual MMR journal/calendar; official patch list/detail; local two-side draft simulator; shareable read-only draft.
Social (M2)	Friends and party membership; pair/trio synergy with sample sizes; shared climb and session recap; multiplayer draft rooms with captain roles and server-enforced turns.
Intelligence (M3)	Personalized patch impact; hero pool change alerts; draft challenge library; rule-based composition feedback; saved experiments and outcomes.
Optional later	Draft bot, replay timelines, advanced ward maps, coaching notes, notifications, paid tier only after licensing and hosting review.

2.1 Match and progression

- Steam ID lookup, shareable profiles, latest matches, result, hero, role if confidently inferred, patch, game mode, region, duration, K/D/A and match deep link.
- Filters: date, patch, ranked/unranked, hero, role, solo/duo/trio/4/5/unknown, with friend, and public/private visibility where known.
- Solo versus party: show games, win rate, rolling trend, hero distribution, and uncertainty. Derive party size from source field when present; otherwise label unknown. Never infer solo from a missing value.

-
- MMR: user enters dated actual MMR snapshots; attribute net change to intervals with clear limitations. Optionally show a separate win/loss-based estimate labeled estimate. No fabricated match-level exact MMR.
 - Sessions: group adjacent matches by configurable inactivity gap; editable notes, goals, and session recap; preserve raw match timestamps.
 - MMR calendar (reference screenshot): week/month/year/all-time navigation; calendar heatmap of daily gains/losses and match count; current MMR, period net gain, win rate, best/worst day and drilldown to matches. Toggle solo/party/overall and filter by account/patch. Exact daily deltas require daily or per-match user observations; otherwise show unknown or clearly labeled estimates. Never imply daily MMR came from match API.

2.2 Group play

- Invite friends by URL or Steam profile; link each member to their own Steam account after consent. Shared match detection uses match IDs and known party membership, not merely appearing on the same team.
- Pair/trio and hero chemistry: win/loss and lane/role combinations, recent patch view, minimum sample thresholds, and comparison against individual baseline with explanatory caveats.
- Shared climb: roster, played-together matches, role lineup, experiment notes, and opted-in MMR entries; private by default.

2.3 Patch hub

- Display official source link, version, publication date, section hierarchy, hero/item/map/general notes, and source language. Preserve original wording and attribution; store normalized structured references only when parsable.
- Watchlist by hero, item and role. Show “changes affecting your pool” and compare before/after patch match cohorts with minimum sample and warning about confounders.
- Daily refresh plus admin manual refresh; idempotent import and content hash. On parser failure, keep official link and raw source reference, not invented summaries.

2.4 Draft practice

- Local practice: two sides, selected patch/ruleset, pick/ban sequence, availability, undo in local mode, timer toggle, save/load, share snapshot.
- Multiplayer: create/invite room, two captains, optional spectators and suggestions, server-validated actions, authoritative timer, reconnect and resync, audit trail, host controls, and rematch.
- Modes: “current Captain’s Mode-style” and custom practice. Store the sequence as versioned configuration and verify live-game order at implementation time; do not hard-code an unverified ban sequence as canonical.
- Feedback: transparent heuristics for stuns, initiation, saves, pushing, damage profile, lane pairing and hero comfort. Give reasons and confidence, not a fake exact win probability.
- Challenges: current-patch opening position, respond to a target lineup, ban priority drills, last-pick puzzle, compare alternate branches, and replay draft decisions.

3. User journeys and acceptance criteria

Journey	Acceptance criterion
Guest explores	Can read public patch pages, look up public player data, and run a local draft without login; privacy and missing-data states are clear.
Steam sign-in	User returns from Steam; server verifies OpenID response and claimed SteamID; one local account is created/upserted; session survives reload; sign-out revokes local session.
Solo/party review	Every imported match has classification solo/party/unknown plus source and confidence; dashboard totals reconcile to filtered matches.
MMR journal	User can add/edit/delete own entries; charts distinguish actual observations from interpolation; multiple accounts never mix.
Draft room	Only active captain can make legal pick/ban; simultaneous actions resolve once; reconnect sees same state; paused/expired timer behavior is deterministic.
Patch update	Duplicate fetch does not duplicate patch; official link and source timestamp shown; partial parse surfaces a degraded status.

4. Architecture: modular monolith

One Next.js deployment owns UI, API routes and server actions. A MongoDB Atlas cluster stores user data and cached normalized external records. Keep integrations behind adapters so API providers can change. Real-time multiplayer needs a managed pub/sub or realtime provider at scale; Mongo polling is acceptable only for a small prototype with explicit limits.

Suggested modules: identity, players, matches, mmr, groups, patches, drafts, challenges, notifications, shared infrastructure. Each owns contracts, application use cases, domain logic, repositories and UI. Dependency flow: route/action → authorization and validation → use case → repository/provider → typed result. UI never imports MongoDB or secrets.

4.1 Example directory layout

```
src/app/(public)/, src/app/(app)/, src/app/api/ — routes and composition only
src/modules/{domain}/domain/ — entities, pure calculations, rules
src/modules/{domain}/application/ — use cases, input/output contracts
src/modules/{domain}/infrastructure/ — Mongo repositories and upstream API adapters
src/modules/{domain}/ui/ — screens and domain components
src/components/ui/ — shadcn components; src/lib/ — configuration, logging, DB, auth
tests/unit/, tests/integration/, tests/e2e/, scripts/, docs/adr/
```

- Use strict TypeScript, explicit return types at service boundaries, Zod parsing for environment and untrusted inputs, discriminated error types, and stable IDs. Avoid generic “utils” or abstract base classes without a use case.
- Use Server Components for initial reads, client components for forms/draft interactions, Route Handlers for external callbacks and API endpoints. Write operations enforce authorization on the server, regardless of UI state.
- Pin a compatible, current stable Next.js/React/Node matrix at project initialization; commit lockfile and document upgrade policy. Do not assume package versions from this document remain current.

4A. Domain-driven design and design patterns

Use pragmatic DDD inside a modular monolith. Define bounded contexts by business capability, not by framework layer. Each context owns its vocabulary and persistence contract; cross-context imports go through public application interfaces. Avoid a giant shared domain model or a repository per trivial object.

Bounded context	Aggregate / invariant
Identity	User and linked SteamAccount: unique SteamID64; only owner can unlink; session is infrastructure.
Match Intelligence	PlayerMatchFact: unique player+match, provenance required, queue classification may be unknown. Match import is idempotent.
Progression	MMRSnapshot and Session: actual value has observation time/source; edits recompute derived views; do not turn estimated delta into actual.
Groups	Group and Membership: owner controls invites, member consent gates private sharing.
Patch Intelligence	Patch: version/source unique, immutable published source plus parse revision; user watchlist separate aggregate.
Drafting	DraftRoom: one authoritative turn, unique hero picks/bans, legal phase, versioned ruleset; DraftEvent append-only.

4A.1 Tactical patterns, applied selectively

- Entities for identity and lifecycle; value objects for SteamID64, PatchVersion, MMRValue and DraftAction; domain services only for rules spanning entities; pure policy functions for queue classification, draft legality and patch assignment.
- Repository interfaces live with application/domain contracts; Mongo implementations live in infrastructure. Use an anti-corruption layer to translate OpenDota/Steam payloads into internal typed facts, with source/provenance preserved.
- Use command/query separation: commands validate authorization and invariants; queries return optimized read models. This is lightweight CQRS within one database, not separate event infrastructure for the entire app.
- Drafting uses a reducer/state machine plus optimistic concurrency and an append-only event log. Rebuild room state from snapshot+events. Do not force event sourcing on users, patches or imported matches.
- Use strategy interfaces for provider adapters and versioned draft rulesets. A factory chooses a ruleset by stored version. Use dependency injection at the route composition boundary with simple constructors; avoid a DI container unless complexity proves the need.
- Outbox-like persisted jobs for critical asynchronous notifications/import updates only if background reliability becomes necessary; no distributed microservices or Kafka for MVP.

4A.2 Context boundaries and consistency

- DraftRoom changes require immediate consistency and one accepted version. Match imports and group summaries can be eventually consistent. Publish internal domain events such as MatchImported, MMRSnapshotRecorded and PatchImported through a thin in-process interface; subscribers update read models idempotently.
- No direct Mongo collection reads across contexts. A dashboard composes query services from Match Intelligence, Progression and Groups. Keep context APIs typed and document them in ADRs.
- Architecture tests enforce imports: domain imports no Next.js/Mongo/provider packages; application depends on domain and ports; infrastructure implements ports; app routes depend on application composition.

5. Identity, accounts, privacy and authorization

- Steam sign-in uses Steam OpenID 2.0 via redirect to Steam Community. Verify return_to, realm, nonce/replay protection and positive provider verification server-side; parse and validate the claimed SteamID64. Never ask for or store a Steam password.
- Choose a maintained, audited Steam OpenID integration compatible with the selected Next.js auth/session library. If no trustworthy provider exists, implement a narrow server-side adapter with integration tests against the protocol; do not label Steam OpenID as generic OAuth.
- Store internal user ID, SteamID64 as a string, optional public persona/avatar snapshot, account created/updated timestamps. Derive account_id/Steam32 using exact integer conversion, never JavaScript floating point arithmetic on SteamID64.
- Session: secure HttpOnly cookie, SameSite=Lax, Secure in production, rotation/expiry; CSRF checks on mutations; server-side ownership checks. Guests may read public data but cannot alter profiles or rooms.
- Roles: user; group owner/member; draft captain/spectator; admin for patch ingestion. Ownership and room membership are resource scoped. Invite tokens are random, expiring, single-purpose and revocable.
- Profile and MMR data private by default, explicit share controls, export/delete account and notes, no public disclosure of imported friend data without consent. Respect upstream privacy and terms.

6. Data sources, trust and refresh

Source	Use and caveat
OpenDota	Public player/match/hero endpoints. Some fields and parsed replay detail can be missing or delayed. Respect API quotas, cache responses and retain source/fetchedAt/parseStatus.
Steam Web API	Optional official match/profile inputs; server-held key if needed. Validate endpoint availability and key terms before adding.
Dota official patch pages	Canonical patch links/content where accessible. Create a provider adapter; treat page parsing as brittle and monitor failures.
User input	Actual MMR snapshots, hero pool, goals, group notes and draft annotations; label user-entered versus provider-derived data.

- Provider gateway: bounded retries with jitter, timeout, rate-limit handling, circuit breaker/light backoff, cache TTL by resource, ETag if available, request deduplication, and per-user/API quota budgets.
- Never expose API keys in client bundle. Use request-specific server context; no global mutable user state. Keep provider JSON as a bounded optional diagnostic payload rather than the primary domain model.
- Classification: party_size=1 means solo if upstream field is trustworthy; greater than 1 means party; null/missing means unknown. Source precedence documented; friends co-occurring on a team does not prove queued party.
- Historical patch assignment based on match start time and effective patch timeline, with uncertain boundary matches labeled accordingly.

7. MongoDB Atlas design

Use MongoDB Atlas Free for prototype. Create one cluster, separate database users and dev/staging/prod databases or clusters where possible. On serverless hosting reuse a cached MongoClient per warm instance, configure a sensible maxPoolSize, and monitor connections. Free tier is limited in storage/throughput and pauses after prolonged inactivity.

Collection	Key fields / indexes
users	<code>_id</code> , <code>steamId64</code> , <code>persona</code> , <code>settings</code> , <code>createdAt</code> ; unique <code>steamId64</code> .
player_accounts	<code>_id</code> , <code>userId</code> , <code>steamId64</code> , <code>accountId32</code> , <code>visibility</code> , <code>lastSyncAt</code> ; unique <code>steamId64</code> ; index <code>userId</code> .
matches	<code>matchId</code> , <code>startedAt</code> , <code>patchId</code> , <code>gameMode</code> , <code>provider</code> , <code>fetchAt</code> ; unique <code>matchId</code> ; date index.
player_match_facts	<code>accountId32</code> , <code>matchId</code> , <code>herold</code> , <code>result</code> , <code>role</code> , <code>partySize</code> , <code>queueClass</code> , <code>confidence</code> ; unique <code>{accountId32,matchId}</code> ; indexes by <code>account/date</code> , <code>patch</code> , <code>queueClass</code> .
mmr_entries	<code>userId</code> , <code>accountId32</code> , <code>observedAt</code> , <code>mmr</code> , <code>source</code> , <code>note</code> ; index <code>{userId,accountId32,observedAt}</code> .
mmr_calendar_daily	Derived <code>accountId32</code> , <code>localDate</code> , <code>timeZone</code> , <code>actualDelta/estimatedDelta</code> , <code>wins/losses</code> , <code>matchCount</code> , <code>basis</code> and <code>lastRebuiltAt</code> ; unique <code>{accountId32,localDate,timeZone}</code> ; rebuild on snapshot edits.
groups / memberships	<code>groupId</code> , <code>ownerId</code> , <code>member userIds</code> , <code>visibility</code> ; unique invite hash if active; index <code>member userId</code> .
patches	<code>version</code> , <code>publishedAt</code> , <code>sourceUrl</code> , <code>contentHash</code> , <code>parseStatus</code> , <code>sections</code> ; unique <code>version/source</code> ; date index.
draft_rooms / events	<code>roomId</code> , <code>patchId</code> , <code>rulesetVersion</code> , <code>stateVersion</code> , <code>status</code> , <code>players</code> ; events keyed <code>{roomId,sequence}</code> ; unique indexes; optional expiry TTL.
session_notes / challenges	<code>owner/group access</code> , <code>matchIds</code> or <code>draftId</code> , <code>notes</code> , <code>goal</code> , <code>timestamps</code> ; indexes by <code>owner</code> and <code>createdAt</code> .

- Choose embedding for small bounded objects (settings, draft summary) and references for large match/event histories. Never put all a player’s matches into a single document.
- Use idempotent upsert on provider imports and unique indexes. Store Date objects in UTC; render user-local time. Add schema version and migration scripts for evolving shapes.
- Draft writes use expected stateVersion with conditional update or transaction where supported; atomically accept one event, reject stale clients and broadcast the committed state. Keep a replayable event log and periodic snapshot.

8. UI and forms

- Use shadcn/ui with Tailwind and accessible primitives; customize tokens for a restrained dark Dota visual language. Include light mode if feasible; keyboard support, visible focus and reduced-motion behavior are requirements.
- Prefer React Hook Form + Zod over Formik for this stack because shadcn/ui documents the integration and schema reuse is straightforward. Use one form strategy across the app; Formik only if the team has a concrete existing dependency.
- Forms: profile visibility, manual MMR entry, group creation/invite, draft setup/rules, hero watchlist, session note, report a classification error. Validate on client for feedback and on server for trust.
- Screens: landing; player search/profile; dashboard; match list/detail; solo vs party; friend/group; shared climb; patch index/detail; watchlist; draft lobby/room/replay; challenge library; settings/data controls.
- Every screen defines loading/empty/error/stale states and data source label. Charts show denominators and sample counts, avoid visually exaggerating small differences, and can be read without color alone.

9. API contracts and background work

Use versioned HTTP routes for browser-accessible resources; mutation handlers return typed errors and enforce idempotency where retried. Suggested endpoints below are illustrative, not a requirement to expose every internal use case.

Route	Purpose
GET /api/v1/players/:id/summary	Public or authorized cached dashboard with source timestamps.
GET /api/v1/players/:id/matches	Cursor pagination; validated filters; capped page size.
POST /api/v1/mmr-entries	Create owned actual MMR snapshot; validate range/date.
GET /api/v1/patches; GET /:version	Index/detail with official links and parse status.
POST /api/v1/drafts	Create versioned room from validated ruleset.
POST /api/v1/drafts/:id/actions	Submit pick/ban with expected version and idempotency key.
GET /api/v1/drafts/:id/events	Resync/poll/SSE depending deployment strategy.

- Sync player matches on demand with cooldown, paginated backfill and per-account lock; incremental updates later. Daily patch refresh is sufficient for MVP, with manual admin retry.
- For live shared drafts, prefer a managed realtime channel/pub-sub plus authoritative persistent state. If avoiding a new provider, use short polling for early prototypes and cap rooms/users; avoid pretending serverless functions hold durable WebSocket state.

10. Security, reliability and operational standards

- Secrets: environment validation; separate dev/staging/prod; .env.example without values; least-privilege Atlas user, allowlisted network where practical, key rotation and no secrets in logs.
- Security: input limits, query allowlists, injection-safe Mongo filters, output encoding, CSRF/session checks, rate limiting for lookup/invite/draft actions, abuse controls and audit logs for admin/room changes.
- Performance: request caching, route revalidation, paginated imports, compound indexes checked with explain plans, client bundle budgets, image optimization and server-side render where it improves first load.
- Operations: structured JSON logs with request ID, provider latency/error metrics, draft conflict rate, import lag, patch parser failure alerts, health endpoint, uptime monitor, backup/export plan and privacy deletion runbook.
- Accessibility: semantic controls, screen-reader status for draft turn/timer, keyboard pick/ban, focus management in dialogs, responsive layout and contrast review.
- Legal/branding: unofficial fan project disclaimer, avoid implying Valve endorsement, attribute official material, review Valve and provider terms before rehosting full patch text/art or commercial use.

11. Testing and quality gates

Layer	Tests / gate
Unit (Vitest)	Pure party classifier including missing fields; Steam ID conversion; MMR calendar timezone and snapshot attribution; patch assignment; draft state machine, timers and illegal actions; heuristic explanations.
Integration	Mongo repository against isolated test DB; unique indexes/idempotent upserts; auth callback verification; OpenDota and patch adapters with recorded/mock responses; stale draft action rejection.
Component	Forms: validation, errors, keyboard interaction, accessible labels; dashboard unknown/empty and sample-size presentation.
E2E (Playwright)	Guest lookup; Steam callback with test identity adapter (never real credentials in CI); create MMR entry; two-client draft race and reconnect; patch failure fallback.
CI gates	Formatting, ESLint, tsc --noEmit, unit/integration tests, production build, dependency/security scan; E2E on preview or scheduled CI.
Release gates	Feature flags for multiuser drafts; migration dry run; smoke test; rollback procedure; source attribution and privacy review.

- Test behavior and invariants rather than snapshots of implementation. Fixture data must be synthetic or anonymized; include timezone boundary, missing match fields, private profile and patch rollover cases.

12. Deployment and cost assumptions

- Next.js on Vercel Hobby is suitable for a small non-commercial prototype. It deploys Route Handlers as functions; free plan has usage limits and Vercel restricts Hobby to non-commercial personal use.
- MongoDB Atlas Free is suitable for a small prototype; review current storage and transfer quotas. Database hosting is separate from the single Next.js codebase.
- Vercel Hobby cron can run at most once daily; use a daily patch refresh and user-triggered match sync. Faster refresh or high-traffic real-time rooms may require paid infrastructure.
- Create a budget dashboard: function invocations/duration, outbound provider requests, database connections/storage/egress, realtime messages and error rate. Alert before quotas are hit.

13. Claude Code execution brief

Instruction to paste: Implement this specification in small, reviewable milestones. First inspect the repo and current official docs, then write an architecture decision record for Steam OpenID/session library, Mongo driver strategy, and multiplayer transport. Scaffold M0 with CI and test infrastructure. Implement M1 vertical slices one at a time: sign-in, import, dashboard, MMR journal, patch hub, local draft. Do not start M2 until M1 acceptance criteria pass. At each milestone, run lint, typecheck, relevant tests and build; summarize changed files, evidence, open risks and next step. Never fabricate MMR, party membership or predictive win rates. Ask only for secrets or external accounts when needed for the next deploy/integration step.

13.1 Definition of done per feature

- User-visible acceptance criteria, server-side authorization and validation, typed adapter/repository boundary, focused tests, empty/error/loading UI, telemetry, documentation, and verified production build.

13.2 Suggested first tickets

- 1. Initialize Next.js TS App Router; design tokens, shadcn/ui, React Hook Form/Zod, lint/format/typecheck/Vitest/Playwright CI.
- 2. Atlas connection singleton, env parser, repositories, indexes, migration and local test DB strategy.
- 3. Steam OpenID identity adapter, session handling, /me and sign-out with callback tests.
- 4. OpenDota adapter, sync service, match normalization and solo/party/unknown classifier with fixtures.
- 5. Player dashboard and filters, manual MMR entry, daily MMR calendar and sample-size-aware charts.
- 6. Patch provider/parser, official links, watchlist, daily refresh and parser failure state.
- 7. Pure draft state machine with configurable ruleset, local draft UI and saved share snapshot.

14. Verification notes and source links

Source check performed 29 September 2026. API and hosting terms can change; recheck before deployment. These sources substantiate integration decisions, not every proposed product feature.

Steamworks authentication: <https://partner.steamgames.com/doc/features/auth>

OpenDota API: <https://docs.opendota.com/>

OpenDota source/types: <https://github.com/odota/core>

Dota official news: <https://www.dota2.com/news>

shadcn/ui React Hook Form guide: <https://ui.shadcn.com/docs/forms/react-hook-form>

MongoDB Atlas Free limits: <https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/>

Vercel Hobby plan: <https://vercel.com/docs/plans/hobby>

Vercel cron scheduling: <https://vercel.com/docs/cron-jobs/manage-cron-jobs>

Vercel Next.js functions: <https://vercel.com/docs/functions/functions-api-reference>